

LAWICEL CANUSB

CAN to USB
Interface Adapter

User Manual

2026-07-12

Document Rev. 1.1-en



Copyright ©2026 by
ELMICRO Computer GmbH & Co. KG
Hohe Str. 9, 04107 Leipzig, Germany
Tel: +49-341-9104810

Tech Support: support@canusb.com
Web: <https://www.canusb.com>

All rights reserved.

Contents

1	Introduction	4
1.1	CANUSB Specifications	5
1.2	Installation	5
1.3	Testing the CANUSB	5
1.4	CANUSB limitations	6
1.5	CAN Driver Design Guide	7
1.6	Version Information	8
2	Serial ASCII Protocol	10
2.1	Set Standard Speed (S)	11
2.2	Set Special Speed (s)	12
2.3	Open (O)	13
2.4	Close (C)	13
2.5	Transmit Standard Frame (t)	14
2.6	Transmit Extended Frame (T)	15
2.7	Transmit Standard RTR Frame (r)	16
2.8	Transmit Extended RTR Frame (R)	17
2.9	Read Status Flags (F)	18
2.10	Set Acceptance Code Register (M)	19
2.11	Sets Acceptance Mask Register (m)	20
2.12	Get Version (V)	22
2.13	Get Serial Number (N)	22
2.14	Enable Time Stamp (Z)	23
3	Document Revision History	25

1 Introduction

The LAWICEL CANUSB is a low cost and easy to use CAN to USB dongle, that could be used together with Windows or Linux O/S with two types of drivers. One Virtual COM Port driver (VCP) which acts as a standard RS232 COM port (in this mode CANUSB can replace our CAN232 and gain more bandwidth on CAN with minor or none software change). The other driver is a Direct Driver (D2XX) which uses a DLL to communicate with the CANUSB. Both drivers can not co-exist on the same PC, you have to choose one of them, however it is easy to do uninstall and change from e.g. VCP to D2XX driver. CANUSB uses same type of commands (standard ASCII format) such as the CAN232 V2, though some old way of communicating (V1 Poll and Poll All) has been removed in favour of new optimized transfer speeds. The CANUSB handles both the 11bit ID format as well as the 29bit ID format, RTR frames, built in FIFO queues, extended info/error information and simple power up through a few commands, no external power is needed, it is supplied from the USB port. The CANUSB is only 60mm long, 35mm wide and 15mm thick using the latest technology of small SMD parts on both sides of the board, the power behind is an Atmel AVR, the Philips SJA1000 CAN controller and the FTDI Parallel FT245RL USB chip and that makes it very flexible in the way of handling small bursts of CAN frames at a high bus speed. This document describes the 1:st version of the CANUSB which can be updated via a bootloader via USB.

1.1 CANUSB Specifications

- Length: 55mm, Width: 36mm, Height: 16mm.
- Blue Transparent Case, 1 meter shielded USB cable.
- Industrial Temp range -40C to +85C.
- USB 2.0 Full Speed, uses FTDI FT245RL.
- Philips SJA1000 CAN Controller running at 16Mhz.
- Philips 82C251 CAN Transceiver (Fully Compatible with ISO 11898-24V).
- Compliant to CAN 2.0A (11bit ID's) and CAN 2.0B (29bit ID's) and supports RTR frames.
- Free Interface DLL and ActiveX controller or low level Ascii Commands via USB driver.
- CANbus connection according to CiA DS102-1 standard.
- Power supplied via USB (5V), so no external power needed.
- CE Approved, ROHS & WEEE compliant.

1.2 Installation

Please look at www.canusb.com for latest driver packages (VCP & D2XX). On the CANUSB site there are also in depth installation instructions of the CANUSB with the drivers. Please select one of the drivers which suite your needs the most. Maybe you just want to replace the CAN232 with a CANUSB, then choose VCP drivers and you are up running quickly. D2XX drivers needs a totally diffrent approach, though is faster in transferring CAN frames. There is no need for any external power supply, since the CANUSB is powered from USB connector (according to USB standard 5V).

1.3 Testing the CANUSB

This test assumes you are using the VCP drivers. Test the CANUSB by installing it to a PC's USB port and install drivers if

necessary according to instructions on previous page under section 1.1. When the CANUSB gets power the 2 LEDs (red & green) will blink rapidly 3 times after it has been installed (about 1 second after it gets connected). Then start Windows Terminal software (or your favourite terminal software) and set it up to any baudrate, 8 databits, no parity, 1 stop bit, also set local echo on so you can see what you type and set the check flag so that it appends a line feed when it receive and end of line. Finally, make sure you have hardware and software handshaking off. Then make sure you are connected and press the RETURN key and it will make a new line, then press V and RETURN and it will print/reply Vhhss, where hh is the hardware version and ss is the software version. Now you know you have full contact with the CANUSB unit and can set it up with a CAN speed and open the CAN port, send and receive frames. The green LED indicates that a CAN frame is successfully sent or received into the CANUSB unit. Note that you must at least have 2 nodes (CANUSB is counted as one node) to send/receive CAN frames and that the CAN cable network is terminated at both ends with 120 ohms over the CANL and CANH lines plus that a proper twisted pair CAN cable is used. The CANUSB is set to accept all frames, so no need to set filters etc. for testing. The CANUSB can also be tested with the sample programs at www.canusb.com.

1.4 CANUSB limitations

There are of course limitations of how many frames the CANUSB can send & receive. Current version (V1011) is tested with a throughput of sending 1000 standard 11bit frames with 8 data bytes at 125kbit CAN bitrate and the VCP drivers. The "bottle neck" is of course the RS232 VCP drivers and the microcontroller not being able to handle more frames per second. So the CANUSB is aimed for low speed CAN networks and works very well with

CAN speeds at 250kbit or less but of course it is usable up to 1 Mbit (but the bus load may not be high at these speeds or e.g. the filter has to be set to accept some of the frames). The CANUSB has software CAN FIFO queues for both sending and reception. These transmit FIFO can handle 8 frames (standard or extended) while the receive FIFO can handle 32 frames (standard or extended). Furthermore the CANUSB has also USB FIFO's built in the hardware, so it can only handle one or two command at a time, meaning before sending the next command to it, you must wait for an answer from the CANUSB unit (OK which is [CR] or Error which is [BELL]).

1.5 CAN Driver Design Guide

The CANUSB doesn't come with a Higher Level CAN driver/parser. Since many commercial development tools provide an RS232 ASYNC LIB (such as Visual Basic, Delphi etc.) it is simple to write a simple program to "talk" to the CANUSB unit via the VCP drivers. The best way is to make a thread that handles all the communication to the CANUSB and puts all messages in FIFO queues or mail boxes depending on your application. Using the D2XX driver has a similar approach, though is faster in transfer and instead of "talking" to a virtual COM port, you have an API (Application Programmer Interface) to a DLL instead which could be used in C, C++, C#, VB or Delphi etc.

If you are migrating from CAN232, the CANUSB do not have the old fashioned P and A commands. Instead incoming CAN frames are sent out at once on USB, this to save bandwidth on the USB side of the driver since the fastest response of USB is 1mS. Replies back from t, T, r and R will also always reply z[CR] or Z[CR] depending on command. Otherwise commands are the same.

Always start each session (when your program starts) with sending 2-3 [CR] to empty any prior comand or queued character in the CANUSB (many times at power up there could be false characters in the queue or old ones that was from a previous session and make sure you wait for CR and/or BELL response back, ther same ammount as you sent CR out), then check the CAN version with V command (to be sure that you have communication with the unit) then set up the CAN speed with s or S command, then open the CAN port with O, then the CANUSB is in operation for both sending and receiving CAN frames. Send frames with the t ot T command and wait for a response back to see it it was placed in the CAN FIFO transmission queue or the queue was full. Incomming frames from the CAN bus will be sent out at ones. Then once in a while send the F command to see if there are any errors (e.g. each 500-1000mS or if you get an error back from the CAN232). If you get to many errors back after sending commands to the unit, send 2-3 [CR] to empty the buffer (and wait for result back), then issue the commands again, if this continue allert the user or application within your program that there is a communication error (e.g. a damaged CAN transceiver or power failure etc.).

The www.canusb.com website offers many sample programs in source code. These programs are free to use or alter to suit your needs.

1.6 Version Information

The version number of CANUSB consists of 2 versions, one for the hardware and one for the software. These two version numbers are combined into one unique version string with 5 characters starting with a V, then 2 characters for hardware and finally 2 characters for software. E.g. version V1010 indicates that it is hardware version 1.0 and software version 0.0. If we update the hardware we will increase version number of the 2 first characters

and if we add or change commands or correct bugs the software version number will increase. To see if your CANUSB supports the commands in this manual, check which version number you have by sending the V command to the CANUSB (see under commands how it works).

2 Serial ASCII Protocol

Note: All commands to the CANUSB must end with [CR] (ASCII=13) and they are CASE sensitive.

2.1 Set Standard Speed (S)

Set bus communication speed using standard CAN bit rates.

This command is only active if the CAN channel is closed.

Sn[CR]

Allowed range for n is 0...8:

S0	Set 10 kbps
S1	Set 20 kbps
S2	Set 50 kbps
S3	Set 100 kbps
S4	Set 125 kbps
S5	Set 250 kbps
S6	Set 500 kbps
S7	Set 800 kbps
S8	Set 1 Mbps

Example: S4[CR]
Set CAN speed to 125 kbps.

Returns: [CR] for OK or [BEL] for ERROR.

2.2 Set Special Speed (s)

Set bus communication speed and timing by direct manipulation of SJA1000 bit timing registers BTR0 and BTR1.

This command is only active if the CAN channel is closed.

sxxyy[CR]

xx BTR0 value in hex

yy BTR1 value in hex

Example: s031C[CR]
 Setup bit timing registers with BTR0=0x03 &
 BTR1=0x1C, establishing a bus speed of 125 kbps.

Returns: [CR] for OK or [BEL] for ERROR.

2.3 Open (O)

Open the CAN channel.

This command is only active if the CAN channel is closed and has been set up beforehand using the S or s command.

O[CR]

Example: O[CR]
 Open the channel

Returns: [CR] for OK or [BEL] for ERROR.

2.4 Close (C)

Close the CAN channel.

This command is only active if the CAN channel is open.

C[CR]

Example: C[CR]
 Close the channel

Returns: [CR] for OK or [BEL] for ERROR.

2.5 Transmit Standard Frame (t)

Transmit a standard (11 bit) CAN frame.

This command is only active if the CAN channel is open.

tiiiidd...[CR]

iii	Identifier in hex (000-7FF)
l	Data length (0-8)
dd	Byte value in hex (00-FF). Numbers of dd pairs must match the data length, otherwise an error occurs.

Example 1: t10021133[CR]
Sends an 11bit CAN frame with ID=0x100, 2 bytes with the value 0x11 and 0x33.

Example 2: t0200[CR]
Sends an 11bit CAN frame with ID=0x20 & 0 bytes.

Returns: z[CR] for OK or BELL (Ascii 7) for ERROR.

2.6 Transmit Extended Frame (T)

Transmit an extended (29 bit) CAN frame.

This command is only active if the CAN channel is open.

Tiiiiiiiidd...[CR]

iiiiiii	Identifier in hex (00000000-1FFFFFFF)
l	Data length (0-8)
dd	Byte value in hex (00-FF). Numbers of dd pairs must match the data length, otherwise an error occurs.

Example: t0000010021133[CR]
Sends a 29bit CAN frame with ID=0x100, 2 bytes with the value 0x11 and 0x33.

Returns: Z[CR] for OK or BELL (Ascii 7) for ERROR.

2.7 Transmit Standard RTR Frame (r)

Transmit an standard RTR (11 bit) CAN frame.

This command is only active if the CANUSB is open.

riiil[CR]

iii Identifier in hex (000-7FF)

l Data length (0-8)

Example: r1002[CR]

Sends an 11bit RTR CAN frame with ID=0x100 and
DLC set to two (2 bytes).

Returns: z[CR] for OK or BELL (Ascii 7) for ERROR.

2.8 Transmit Extended RTR Frame (R)

Transmit an extended RTR (29 bit) CAN frame.

This command is only active if the CANUSB is open.

Riiiiiiil[CR]

iiiiiii Identifier in hex (00000000-1FFFFFFF)

l Data length (0-8)

Example: t000001002[CR]

Sends an 11bit RTR CAN frame with ID=0x100 and
DLC set to two (2 bytes).

Returns: Z[CR] for OK or BELL (Ascii 7) for ERROR.

2.9 Read Status Flags (F)

Read Status Flags.

This command is only active if the CAN channel is open.

F[CR]

Example: F[CR]
Read Status Flags.

Returns: An F with 2 bytes BCD hex value plus CR (Ascii 13) for OK. If CAN channel isn't open it returns BELL (Ascii 7). This command also clear the RED Error LED. See available errors below. E.g. F01[CR]

Bit 0	CAN receive FIFO queue full
Bit 1	CAN transmit FIFO queue full
Bit 2	Error warning (EI), see SJA1000 datasheet
Bit 3	Data Overrun (DOI), see SJA1000 datasheet
Bit 4	Not used.
Bit 5	Error Passive (EPI), see SJA1000 datasheet
Bit 6	Arbitration Lost (ALI), see SJA1000 datasheet *
Bit 7	Bus Error (BEI), see SJA1000 datasheet **

* Arbitration lost doesn't generate a blinking RED light!

** Bus Error generates a constant RED light!

2.10 Set Acceptance Code Register (M)

Sets Acceptance Code Register (ACn Register of SJA1000).

This command is only active if the CAN channel is closed, but has been initiated using the S or s command.

Mxxxxxxxx[CR]

xxxxxxxx Acceptance Code in hex with LSB first, AC0, AC1, AC2 & AC3.
For more info, see Philips SJA1000 datasheet.

Example: M00000000[CR]
Set Acceptance Code to 0x00000000
This is default when power on, i.e. receive all frames.

Returns: [CR] for OK or [BEL] for ERROR.

2.11 Sets Acceptance Mask Register (m)

Sets Acceptance Mask Register (AMn Register of SJA1000).

This command is only active if the CAN channel is closed, but has been initiated using the S or s command.

mxxxxxxxx[CR]

xxxxxxxx Acceptance Mask in hex with LSB first, AM0, AM1, AM2 & AM3.
For more info, see Philips SJA1000 datasheet.

Example: mFFFFFFFF[CR]
Set Acceptance Mask to 0xFFFFFFFF
This is default when power on, i.e. receive all frames.

Returns: [CR] for OK or [BEL] for ERROR.

Setting Acceptance Code and Mask registers

The Acceptance Code Register and the Acceptance Mask Register works together and they can filter out 2 groups of messages. For more information on how this work, look in the SJA1000 datasheet. In 11bit IDs it is possible to filter out a single ID this way, but in 29bit ID's it is only possible to filter out a group of ID's. The example below will set a filter to only receive all 11bit ID's from 0x300 to 0x3FF.

Commands	Comments
M00006000[CR]	AC0=0x00 AC1=0x00 AC2=0x60 AC3=0x00
m00001FF0[CR]	AM0=0x00 AM1=0x00 AM2=0x1F AM3=0xF0

The first command tells the filter 2 to match 2 bits and if they are not set (in this case it corresponds to 0x3nn, the 3). The second command tells the nn to be don't care, so it could be from 0 to FF, though not so easy to read, since they are not placed nice in a row in memory. Filter 1 s turned off (uses AM0, AM1 & half lower AM3). The last byte in the mask could also be 0xE0 instead of 0xF0, then we filter out the RTR bit as well and you wont accept RTR frames.

2.12 Get Version (V)

Get Version number of both CANUSB hardware and software.

This command is always active.

V[CR]

Example: V[CR]
 Get Version numbers

Returns: V and a 2 bytes BCD value for hardware version and
 a 2 byte BCD value for software version plus CR
 (Ascii 13) for OK. E.g. V1013[CR]

2.13 Get Serial Number (N)

Get Serial number of the CANUSB unit.

This command is always active.

N[CR]

Example: N[CR]
 Get Serial number

Returns: N and a 4 bytes value for serial number plus CR
 (Ascii 13) for OK. E.g. NA123[CR] Note that the se-
 rial number can have both numerical and alfa nu-
 merical values in it. The serial number is also printed
 on the CANUSB for a quick reference, but could e.g.
 be used in a program to identify a CANUSB so the
 program know that it is set up in the correct way (for
 parameters saved in EEPROM).

2.14 Enable Time Stamp (Z)

Enable or disable adding a time stamp to frames received.

This command is only active if the CAN channel is closed.

Zn[CR]

The value will be saved in EEPROM and remembered next time the CANUSB is powered up. This command shouldn't be used more than when you want to change this behaviour. It is set to OFF by default, to be compatible with old programs written for CANUSB. Setting it to ON, will add 4 bytes sent out from CANUSB with the A and P command or when the Auto Poll/Send feature is enabled.

When using Time Stamp each message gets a time in milliseconds when it was received into the CANUSB, this can be used for real time applications for e.g. knowing time inbetween messages etc. Note however by using this feature you maybe will decrease bandwith on the CANUSB, since it adds 4 bytes to each message being sent (specially on VCP drivers).

If the Time Stamp is OFF, the incomming frames looks like this:
t10021133[CR] (a standard frame with ID=0x100 & 2 bytes)

If the Time Stamp is ON, the incomming frames looks like this:
t100211334D67[CR] (a standard frame with ID=0x100 & 2 bytes)

Note the last 4 bytes 0x4D67, which is a Time Stamp for this specific message in milliseconds (and of course in hex). The timer in the CANUSB starts at zero 0x0000 and goes up to 0xEA5F before it loop around and get's back to 0x0000. This corresponds to exact 60,000mS (i.e. 1 minute which will be more than enough in most systems).

- Example 1: Z0[CR]
Turn OFF the Time Stamp feature (default).
- Example 2: Z1[CR]
Turn ON the Time Stamp feature.
- Returns: [CR] for OK or [BEL] for ERROR.

3 Document Revision History

2026-07-12	V1.1	initial edition converted to $\text{\LaTeX}$, based on the original CANUSB Manual V1.0D published November 2011 by LAWICEL AB
------------	------	--